

Generalización para número arbitrario de procesos

Nuria Guerra Casal Adrian Gijon Yubero
Grao en Enxenaria Informatica

1. Introducción

Este ejercicio generaliza la solución del apartado 2 (`prod_sem.c` y `cons_sem.c`) para soportar un **número arbitrario de productores y consumidores**, elegido por el usuario al lanzar el programa. El requisito fundamental es que el **total de caracteres producidos sea igual al total de caracteres consumidos**, independientemente de cuantos procesos participen en cada rol.

La solución mantiene un único buffer compartido entre todos los procesos y los mismos tres semáforos del apartado 2. El semáforo `MUTEX` sigue garantizando la exclusión mutua incluso cuando varios productores o consumidores compiten por acceder al buffer al mismo tiempo, sólo uno de los $N + M$ procesos puede estar dentro de la región crítica.

2. Garantía de equivalencia

El mecanismo que garantiza la equivalencia es calcular automáticamente cuántas iteraciones debe realizar cada consumidor en función del número de productores y consumidores:

$$\text{iter_cons} = \frac{N \times \text{TOTAL_ITERS}}{M}$$

donde N es el número de productores, M el de consumidores y `TOTAL_ITERS`=80. El programa comprueba al inicio que $N \times 80$ sea divisible entre M y termina con un mensaje de error si no lo es, evitando ejecuciones con datos inconsistentes:

```
1 int total = num_prod * TOTAL_ITERS;
2 if (total % num_cons != 0) {
3     fprintf(stderr, "Erro, N*TOTAL_ITERS (%d) non e divisible entre M (%d)\n",
4         total, num_cons);
5     return EXIT_FAILURE;
6 }
7 int iter_cons = total / num_cons; //cada consumidor consume exactamente esto
```

Listing 1: Verificación de divisibilidad y cálculo de iteraciones por consumidor

De esta forma, la suma de todos los caracteres producidos ($N \times 80$) siempre coincide con la suma de todos los consumidos ($M \times \text{iter_cons}$).

3. Estructura

El proceso padre inicializa el buffer compartido y los tres semáforos antes de crear ningún proceso hijo. A continuación lanza los N productores y los M consumidores mediante `fork()`, conservando los `pid` de cada uno para poder esperar su finalización con `waitpid()`. Cada proceso hijo abre los semáforos ya existentes sin reinicializarlos, con `sem_open(..., 0)`.

Para que productores distintos lean partes distintas del fichero, cada productor avanza tantas posiciones como su identificador al abrir el archivo:

```

1 // El productor 'id' salta 'id' caracteres al inicio del fichero
2 for (int i = 0; i < id; i++) {
3     if (fgetc(f) == EOF){
4         rewind(f);
5     }
6 }

```

Listing 2: Desplazamiento inicial para lectura intercalada entre productores

Cada productor y cada consumidor cuentan sus propias vocales e imprimen el conteo al finalizar.

4. Sincronización con semáforos

Las funciones `insert_item()` y `remove_item()` son idénticas a las del apartado 2 en cuanto al protocolo de semáforos. La única diferencia es que ahora reciben los punteros a semáforos como parámetros, porque cada proceso hijo los abre de forma independiente. El mutex garantiza que ninguno de los $N + M$ procesos solape su acceso al buffer:

```

1 sem_wait(vacias); //espera hueco libre (bloquea si buffer lleno)
2 sem_wait(mutex); //solo un proceso a la vez en la region critica
3     datos->pila[datos->cima] = c;
4     datos->cima++;
5 sem_post(mutex); //libera la region critica
6 sem_post(cheas); //avisa a los consumidores

```

Listing 3: Protocolo de inserción con exclusión mutua

5. Velocidades y fases

Productores y consumidores adaptan su velocidad en tres fases mediante `sleep()`s colocados fuera de la región crítica:

Iteraciones	Productores	Consumidores
0 – 29	Sin sleep (rapidos)	Sleep 3s (lentos) → buffer se llena
30 – 59	Sleep 3s (lentos)	Sin sleep (rapidos) → buffer se vacia
60 – 79	Sleep aleatorio 0–3s	Sleep aleatorio 0–3s

Con múltiples productores, el efecto de la primera fase es más intenso: varios productores insertan sin pausa simultaneamente y el buffer se llena aun más rápido que con uno solo.

6. Ejecución

```

# Compilar
gcc -o multiple multiple.c

# 1 productor y 1 consumidor (equivale al apartado 2)
./multiple 1 1 texto.txt

# 2 productores y 4 consumidores (total=160, cada consumidor consume 40)
./multiple 2 4 texto.txt

# 4 productores y 2 consumidores (total=320, cada consumidor consume 160)
./multiple 4 2 texto.txt

```

No es necesario abrir múltiples terminales: el programa lanza todos los procesos internamente y espera su finalización. Al terminar, el proceso padre elimina los semáforos con `sem_unlink()` y libera la memoria compartida con `munmap()`.

7. Resultados y conclusiones

Ejecutando con 2 productores y 4 consumidores se observa que los mensajes [Productor 0] y [Productor 1] aparecen entremezclados en la salida, confirmando que ambos productores estan activos de forma concurrente. Del mismo modo, los cuatro consumidores retiran caracteres de forma intercalada. Durante la primera fase el buffer se llena rápidamente y los productores quedan bloqueados en `sem_wait(vacias)`; durante la segunda fase el buffer se vacía y los consumidores quedan bloqueados en `sem_wait(cheas)`.

```
[Productor 0] Le 'e'
Retirado 't' de pila[9] cima=9 | Pila=[ste es us-]
[Consumidor 1] Consume 't'
Insertado 'e' en pila[9] cima=10 | Pila=[ste es use]
[Productor 1] Le 'm'
Retirado 'e' de pila[9] cima=9 | Pila=[ste es us-]
[Consumidor 0] Consume 'e'
Insertado 'e' en pila[9] cima=10 | Pila=[ste es use]
[Productor 0] Le ' '

```

Figura 1: Captura de parte de ejecución del programa en terminal.

El resultado más importante es que la **suma total de vocales contadas por todos los productores coincide con la suma total de las contadas por todos los consumidores**, confirmando que ningún carácter se ha perdido ni duplicado y que el mutex ha funcionado correctamente con $N + M$ procesos compitiendo por el mismo buffer.

```
[Consumidor 2] Rematou. Vogais: a=0 e=4 i=0 o=3 u=5
[Consumidor 3] Rematou. Vogais: a=0 e=9 i=2 o=6 u=3
[Consumidor 1] Rematou. Vogais: a=0 e=3 i=0 o=4 u=2
[Consumidor 0] Rematou. Vogais: a=0 e=4 i=2 o=7 u=2

[Productor 0] Rematou. Vogais: a=0 e=10 i=2 o=10 u=6
[Productor 1] Rematou. Vogais: a=0 e=10 i=2 o=10 u=6

```

La conclusión es que el mecanismo de tres semáforos del apartado 2 **escala directamente** a cualquier número de productores y consumidores sin modificar el protocolo de sincronización. El semáforo **VACIAS** gestiona automáticamente el bloqueo de cualquier productor cuando el buffer está lleno, **CHEAS** gestiona el bloqueo de cualquier consumidor cuando está vacío, y el **MUTEX** garantiza en todo momento que sólo uno de los $N + M$ procesos accede al buffer. Un cambio adicional respecto al apartado 2 es calcular `iter_cons` para garantizar la equivalencia entre el total producido y el total consumido.